

Введение в объектно-ориентированное программирование (ООП)

1. Члены класса

Основная идеология ООП – объединение в один объект набора некоторых свойств (поля, атрибуты) и поведения (методы, функции).

Класс – это тип данных, определяемый программистом, включающий **атрибуты** (поля класса) и **методы** (функции) для обработки этих данных. Атрибуты и методы вместе называются **членами** класса.

Экземпляр (объект) класса – переменная типа класса.

Пример 1. Класс «Матрица»

```
1  class Matrix
2  {
3  public:
4      int n, m;
5      float** values;
6      Matrix();
7      Matrix(const int &n, const int &m);
8      Matrix(const int &n, const int &m, float** &values);
9      ~Matrix();
10 };
```

Пример 1 должен был напомнить уже известный вам формат определения пользовательских типов – структуру. Аналогично структуре, объявление (стр. 1) начинается с ключевого слова (в данном случае «class»), затем идёт имя (по уже знакомой стилистике – с большой буквы), затем тело класса и в конце не забываем точку с запятой.

Методы могут быть определены как в классе, так и вне его. При определении вне класса используется оператор разрешения области видимости (: :). Обращение к атрибутам и методам вне класса осуществляется через имя объекта и операцию «.» или через имя указателя на объект и операцию «->».

В отличие от структур, тело класса содержит функции. Некоторые из них имеют нетипичное объявление – у них отсутствует тип возвращаемого значения, а название совпадает с названием класса. Это функции – конструкторы (стр. 6-8) и деструктор (стр. 9). Их назначение вам уже знакомо с одной из лабораторных работ, где мы писали их для списка, стека и очереди.

Конструктор – специальный блок операторов (инструкций), вызываемый при создании объекта. Назначение: присвоение начальных значений полям (инициализация), выделение памяти, открытие файлов, сетевых соединений и т.п. Название конструктора всегда совпадает с названием класса.

Деструктор – специальный блок операторов (инструкций), служащий для деинициализации объекта (освобождение памяти, закрытие файлов и т.п.). Название деструктора состоит из названия класса и тильды (~) перед ним. В отличие от функций-«деструкторов» из предыдущего семестра, в реальных деструкторах нет необходимости

«зачищать» поля. Более того, это совершенно бессмысленно, поскольку после выполнения деструктора объект перестаёт существовать.

Конструкторы и деструкторы не вызываются пользователем, как другие методы класса. Они вызываются автоматически при создании и уничтожении объекта.

Конструкторы бывают нескольких типов:

1. **Конструктор по умолчанию** (без параметров) – конструктор, создаёт объекты без пользовательских значений инициализации (со значениями по умолчанию) и необходимы для объявления переменных типа класса. Конструктор без параметров может быть только один. Если вы не пишете ни одного конструктора по умолчанию или с параметрами, то компилятор сгенерирует конструктор по умолчанию за вас. Этот конструктор будет пуст, он не будет инициализировать члены класса (там будет лежать мусор), но его наличие позволит создавать объекты. Однако если вы определили конструктор с параметрами, то конструктор по умолчанию сгенерирован не будет.

Пример 2. Конструктор по умолчанию

```
1 Matrix::Matrix()
2 {
3     n = m = -1;
4     values = nullptr;
5 }
```

2. **Конструктор с параметрами** – конструктор, который принимает один или несколько параметров. Таких конструкторов может быть сколько угодно, но, как и любые перегрузки функций, список параметров (по кол-ву и/или типу) для каждого конструктора должен быть уникален.

Пример 3. Конструктор с параметрами

```
1 Matrix::Matrix(const int &n1, const int &m1)
2 {
3     n = n1;
4     m = m1;
5     values = new float*[n];
6     for (int i = 0; i < n; ++i)
7         values[i] = new float[m];
8 }
```

Пример 4. Конструктор с параметрами

```
1 Matrix::Matrix(const int &n1, const int &m1, float** &values1)
2 {
3     n = n1;
4     m = m1;
5     values = values1;
6     values1 = nullptr;
7 }
```

В строке 6 примера 4 меняется значение указателя `values1`, чтобы данные, переданные в класс, не были испорчены извне.

3. **Конструктор копирования** – конструктор, который используется для создания нового объекта через копирование существующего. Если вы не определяете конструктор копирования, компилятор его сделает за вас. В этом конструкторе будет почленная инициализация (т.е. каждому члену будет присвоен его аналог в исходном объекте), что приведёт к неожиданным результатам при наличии в классе членов, использующих динамическое выделение памяти. Например, при почленном копировании матрицы из примера новая матрица-копия будет иметь поле `values`, которое указывает на ту же область памяти, что `values` в исходной матрице. Таким образом, если внести изменения в исходную матрицу, поменяется и копия. Кроме того, выделенная однажды память, будет освобождаться дважды.

Пример 5. Неправильная реализация конструктора копирования (почленное копирование)

```
1  Matrix::Matrix(const Matrix &matrix)
2  {
3      n = matrix.n;
4      m = matrix.m;
5      values = matrix.values;
6  }
```

Пример 6. Правильная реализация конструктора копирования (глубокое копирование)

```
1  Matrix::Matrix(const Matrix &matrix)
2  {
3      n = matrix.n;
4      m = matrix.m;
5      if (matrix.values != nullptr)
6      {
7          values = new float*[n];
8          for (int i = 0; i < n; ++i)
9          {
10             values[i] = new float[m];
11             for (int j = 0; j < m; ++j)
12                 values[i][j] = matrix.values[i][j];
13         }
14     }
15 }
```

4. **Конструктор перемещения** – конструктор, который перемещает ресурсы исходного объекта в копию. Т.к. происходит перемещение ресурсов, а не копирование, то указатели на исходные ресурсы затираются (стр. 6), чтобы при уничтожении исходного объекта память не была очищена. Это одно из проявлений механизмов, обобщённо называемых «семантикой перемещения». Подробно вы будете их изучать на втором курсе.

Пример 7. Конструктор перемещения

```
1  Matrix::Matrix(Matrix &&matrix)
2  {
3      n = matrix.n;
4      m = matrix.m;
5      values = matrix.values;
6      matrix.values = nullptr;
7      std::cout << "Moving\n";
8  }
```

Т.к. мы привыкли копировать и переписывать значения оператором присваивания (=), с конструкторами копирования и конструктором перемещения тесно связаны перегрузки оператора присваивания: оператор присваивания копированием и оператор присваивания перемещением.

Пример 8. Оператор присваивания копированием

```
1  Matrix& Matrix::operator=(const Matrix &matrix)
2  {
3      if (this != &matrix)
4      {
5          if (values != nullptr)
6          {
7              for(int i = 0; i < n; ++i)
8                  delete[] values[i];
9              delete[] values;
10         }
11         n = matrix.n;
12         m = matrix.m;
13         if (matrix.values == nullptr)
14             values = nullptr;
15         else
16         {
17             values = new float*[n];
18             for (int i = 0; i < n; ++i)
19             {
20                 values[i] = new float[m];
21                 for (int j = 0; j < m; ++j)
22                     values[i][j] = matrix.values[i][j];
23             }
24         }
25     }
26     return *this;
27 }
```

В данном примере (стр. 3) используется `this` – скрытый член, который есть у **каждого** класса, где хранится указатель на адрес памяти, по которому находится данный экземпляр класса.

`this` используется тогда, когда метод класса должен вернуть сам объект, а также может использоваться для явного обращения к члену класса, когда его имя совпадает с именем локальной переменной (ситуация не очень хорошая, её стоит избегать, т.к. обилие `this->` ухудшает читаемость кода).

Пример 9. Конструктор с параметрами с обращением через указатель `this`

```
1  Matrix::Matrix(const int &n, const int &m)
2  {
3      this->n = n;
4      this->m = m;
5      values = new float*[this->n];
6      for (int i = 0; i < this->n; ++i)
7          values[this->n] = new float[this->m];
8  }
```

Может возникнуть вопрос, зачем оператору присваивания что-то возвращать, ведь его задача – изменить значения в объекте, стоящем слева от него, а возвращать-то ничего не нужно.

А логика здесь точно такая же, как с перегрузками >> и << для потоков ввода и вывода – чтобы работали «цепочки» (например, `a = b = c;`).

Оператор присваивания перемещением также относится к семантике перемещения. Так что сейчас исключительно ознакомительный пример.

Пример 10. Оператор присваивания перемещением

```
1 Matrix& Matrix::operator=(Matrix &&matrix)
2 {
3     values = matrix.values;
4     n = matrix.n;
5     m = matrix.m;
6     matrix.values = nullptr;
7     matrix.n = matrix.m = -1;
8     return *this;
9 }
```

Когда и что нужно определять для своего класса?

Правило трёх. Если классу требуется пользовательский деструктор (1), пользовательский конструктор копирования (2) или пользовательский оператор присваивания копированием (3), то почти наверняка ему требуются все три.

Правило пяти. Если класс, использующий пользовательский деструктор (1), конструктор копирования (2) или оператор присваивания копированием (3), требует функционал перемещения, то должны быть явно определены конструктор перемещения (4) и оператор присваивания перемещением (5).

Правило нуля. Если класс использует автоматически генерируемый деструктор, автоматически генерируемые конструктор копирования и конструктор перемещения, а также автоматически определённые операторы присваивания копированием и перемещением, то такой класс должен работать только с владением ресурсами (не должно быть динамического выделения памяти, открытия файлов и т.п.).

2. Доступ к членам класса

Вернёмся к самому первому примеру. Там осталось ещё одно незнакомое слово – «`public`». Это модификатор (спецификатор) доступа к членам класса.

Существуют **три** модификатора доступа:

- `public` – полностью доступны как в классе, так и за пределами класса;
- `private` – доступны внутри класса и дружественным функциям и классам (о них поговорим в другой раз);
- `protected` – доступны только в своем классе, производных и дружественных классах и функциях.

Модификатор доступа распространяется на все члены класса, которые объявлены после модификатора. В первом примере все члены класса объявлены публичными. Для реальной

программы это, разумеется, не должно быть так. В большинстве случаев публичными должны быть конструкторы (иначе в основной программе нельзя будет создать объект) и другие методы, которые позволяют работать с экземплярами класса. Приватными остаются методы и атрибуты, которые **используются только внутри класса и не должны быть доступны извне**. Более близкий к реальному – пример 11.

К приватным полям класса можно обеспечить «однонаправленный» доступ. Для этого используются публичные методы «геттеры» (от англ. *get* – получать) и «сеттеры» (от англ. *set* – задавать), которые позволяют производить соответствующие действия. Т.к. методы являются членами класса, они могут обращаться к полям класса, а их «публичность» позволяет через них получить доступ к значениям полей извне. Также в сеттеры могут быть включены проверки входных данных. Таким образом, программист вместо прямого обращения к полям использует геттеры для получения значений полей и сеттеры для изменения их значений.

Пример 11. Класс-матрица

```
1  class Matrix
2  {
3  private:
4      int n, m;
5      float** values;
6  public:
7      Matrix(); // конструктор по умолчанию
8      Matrix(const int &n, const int &m); // конструктор с параметрами
9      Matrix(const int &n, const int &m, const float** &values); // конструктор
10 с параметрами
11      Matrix(const Matrix &matrix); // конструктор копирования
12      Matrix(Matrix &&matrix); // конструктор перемещения
13      float det(); // функция вычисления определителя
14      Matrix& transpose(); // транспонирование матрицы
15      int getRows() // получить кол-во строк
16      {
17          return n;
18      }
19      int getColumns() // получить кол-во столбцов
20      {
21          return m;
22      }
23      void setRows(int n) // задать кол-во строк
24      {
25          this->n = n;
26      }
27      void setColumns(int m) // задать кол-во строк
28      {
29          this->m = m;
30      }
31      void fillValues(); // заполнить матрицу
32      void print(); // вывести матрицу на экран
33      Matrix& operator=(const Matrix &matrix); // оператор присваивания
34 копированием
35      Matrix& operator=(Matrix &&matrix); // оператор присваивания перемещением
36      Matrix operator+(const Matrix &matrix); // сложение матриц
37      ~Matrix(); // деструктор
38 };
```

Важный момент: контроль доступа обеспечивается на уровне **класса**, а не объекта. Это значит, что внутри метода, вызванного конкретным экземпляром класса, можно получить доступ не только к значениям членов этого экземпляра класса, но и других экземпляров класса (например, «приватизация» `n`, `m` и `values` не мешает работе конструкторов копирования и перемещения).

3. Отличие классов от структур

Фактически отличие класса от структуры заключается в модификаторе доступа по умолчанию. Для класса он `private`, для структуры – `public`.

Несмотря на то, что вы можете записать функции в структуру, это будет идеологически неверно и я, как и в прошлом семестре, когда вы знакомились со структурами, ***крайне не рекомендую это делать.***

Используйте структуры, когда вам нужно «объединить несколько переменных в одну», и классы, когда помимо объединения данных вам необходимо заложить в объект некоторый функционал.

4. Инициализация полей класса

При инициализации полей в классе по возможности используйте список инициализации.

Пример 12. Конструктор по умолчанию из примера 4, оформленный через список инициализации

```
1 Matrix::Matrix(const int &n1, const int &m1, const float** &values1)
2     : n(n1), m(m1), values(values1)
3 {
4     values1 = nullptr;
5 }
```

Список инициализации ввиду длины строки вынесен на строку 2. При малой длине строки 1 список инициализации является продолжением строки 1.